Penetration Testing Lab Report

Web Application SQL Injection & Authentication Bypass Assessment

Project Title: Employee Portal SQL Injection Investigation & Flag Extraction

Target Application: StaffHub (localhost:8080)

Author / Tester: Sudip Bastola

Assessment Date: August 10, 2026

Assessment Type: Controlled Academic / Lab Web Application Security Assessment

1. Vulnerability Information

Vulnerability Name	**SQL Injection (Authentication Bypass & Data Exfiltration)**
CWE ID	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
OWASP Category	OWASP Top 10:2021 – A03: Injection
Severity	High **Justification:** Allowed an unauthenticated attacker to bypass authentication mechanisms, gain administrative portal access, and read sensitive user credentials directly from the backend database.
Vulnerable Locations	**1. Main Login Endpoint:** http://localhost:8080/index.php **- Vulnerable Parameter:** username 2. **Employee Profile Endpoint:** http://localhost:8080/profile.php **- Vulnerable Parameter:** id

2. Description

SQL injection is a web application vulnerability that occurs when user-supplied input is directly concatenated or concatenated improperly into a backend SQL query string. Because the application fails to separate code from data, the database engine interprets the user input as executable SQL commands rather than plain text.

During the assessment of StaffHub, two separate parameters were identified as vulnerable to SQL injection. On the main login page (index.php), unvalidated user input in the username field was concatenated into the authentication query. By supplying a crafted payload, the expected authentication logic was altered, allowing an unauthorized user to bypass the password check entirely.

On the employee profile page (profile.php), the numeric id URL parameter was directly inserted into a backend database query used to fetch profile data. By using ORDER BY clause testing, the column counts for both underlying queries were determined. Using UNION SELECT operations, arbitrary data was retrieved from the database. This vulnerability is dangerous because it bypassed core security controls, exposed sensitive account credentials, and allowed full takeover of an administrative user account.

3. Tools Used

Brave Browser : Used to access the StaffHub web application, submit SQL injection payloads through the login and profile parameters, navigate the employee and administrative portals, and observe application/database responses.

Step 1 - Initial SQL Injection Testing on Login Page

Action: Tested the username input field on the main login page (index.php) using incremental ORDER BY logic while monitoring application behavior.

Reason: In SQL, the ORDER BY clause sorts returned records by specified column indexes (e.g., ORDER BY 1 sorts by the first column). By incrementing this integer value, an evaluator can determine the exact number of columns returned by the underlying query when the application produces an error or alters its visual response.

Payload: Submitted incremental integer values in the username field during login attempts (e.g., ORDER BY 1, ORDER BY 2, up through ORDER BY 6 and ORDER BY 7).

Actual Payload: ' ORDER BY n-- -

Where n=1,2,3,4,5,6,7

Explanation: The ORDER BY clause tests how many columns exist in the primary query. If a specified position number exceeds the actual number of columns selected by the database, an database error occurs.

Observation/Result: The application functioned normally up to position 6. When testing position 7, the application returned a database error, confirming that the authentication query returns exactly six columns and that the parameter is vulnerable to SQL injection.

Screenshot:

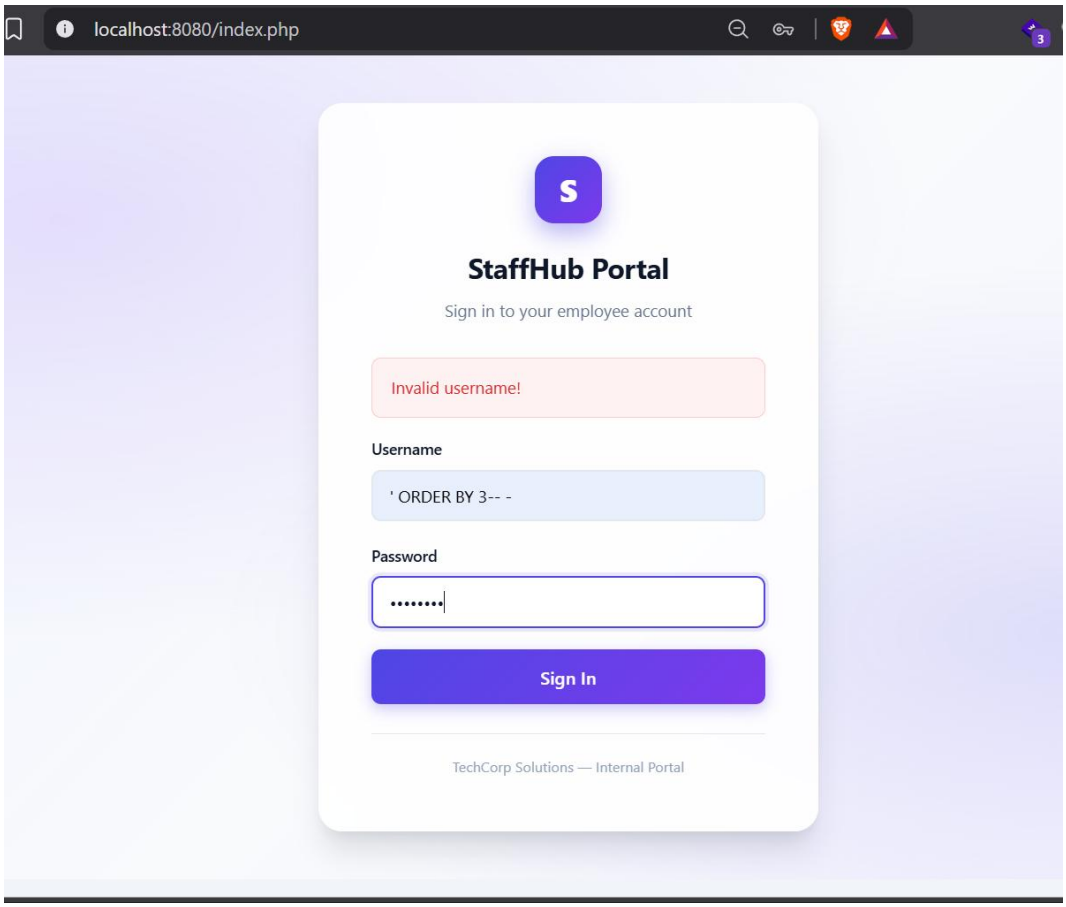

Figure 1 SQL Injection Testing Using ORDER BY : Invalid Username Response

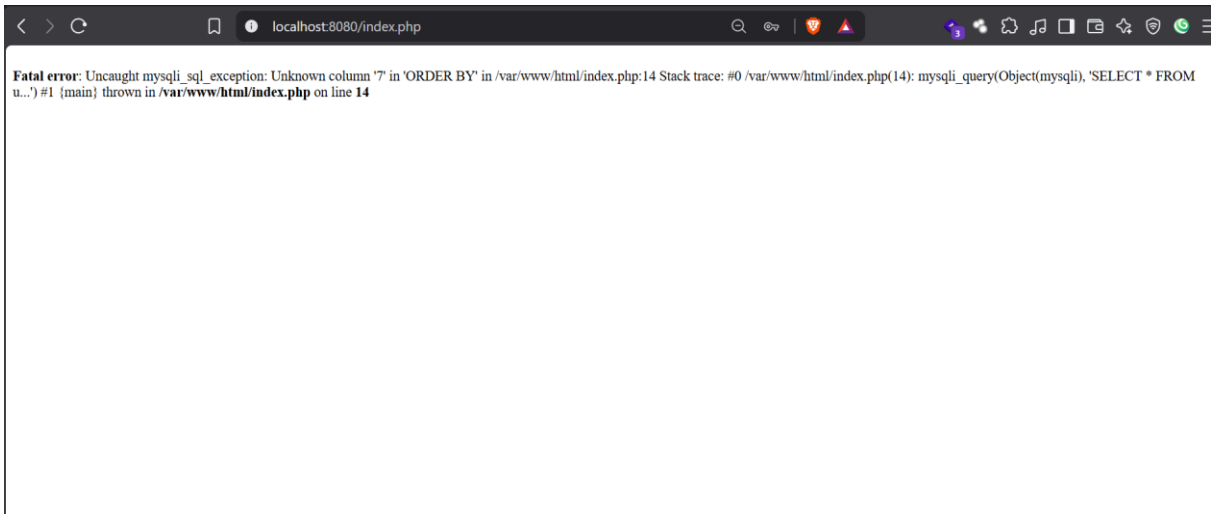

Figure 2 ORDER BY Column Enumeration : Database Error After Exceeding the Valid Column Count

Step 2 - Authentication Bypass

Action: Submitted a targeted UNION SELECT payload into the username field on index.php to bypass the authentication check.

Reason: The goal was to force the application to accept a dummy database row containing known values, allowing login without a valid user password.

Password:mypassword
Username Payload: ' UNION SELECT 'admin', NULL, 'mypassword', 'admin', NULL, NULL-- -

Explanation: Breakdown of the payload components:

- **' (Single Quote):** Closes the original string parameter context in the SQL query.

- **UNION SELECT:** Combines the results of the original query with a secondary, injected query. The number of expressions provided must match the six columns expected by the query.

- **'admin':** Placed into column positions expected to handle user account identifiers, supplying a mock administrative username to the application logic.

- **'mypassword':** Placed into the position corresponding to the password hash or plain-text field, matching the password value entered in the HTTP form.

- **NULL:** Used as placeholder values for the remaining positions whose exact data types or contents are not required for successful authentication.

• **-- -:** An SQL comment sequence that instructs the database server to ignore the remainder of the original query structure.

Observation/Result: The backend application accepted the injected record, authenticated the session as admin, and granted access to the internal employee portal.

Screenshot:

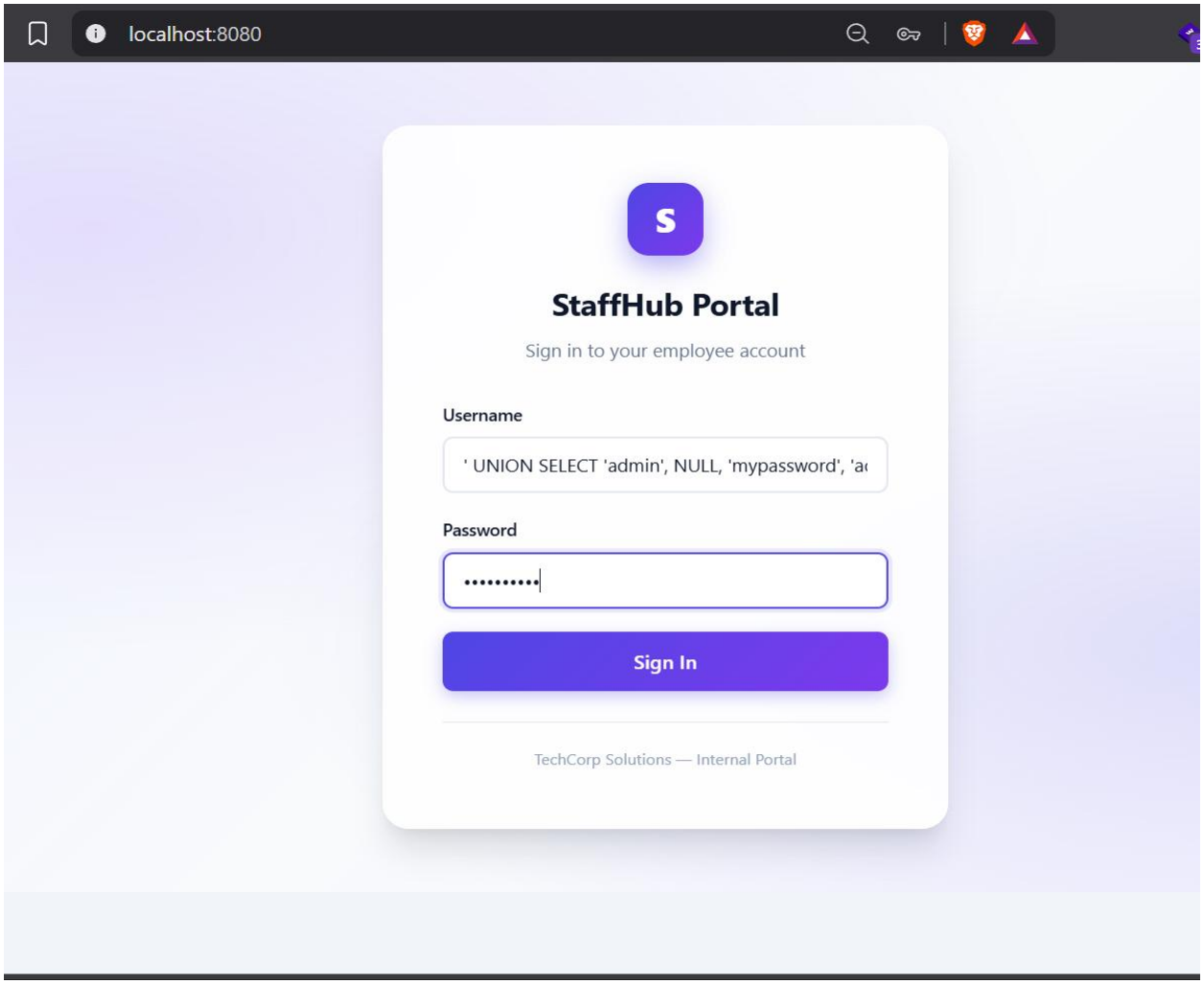

Figure 3 Adding UNION SELECT payload leading to employee portal access

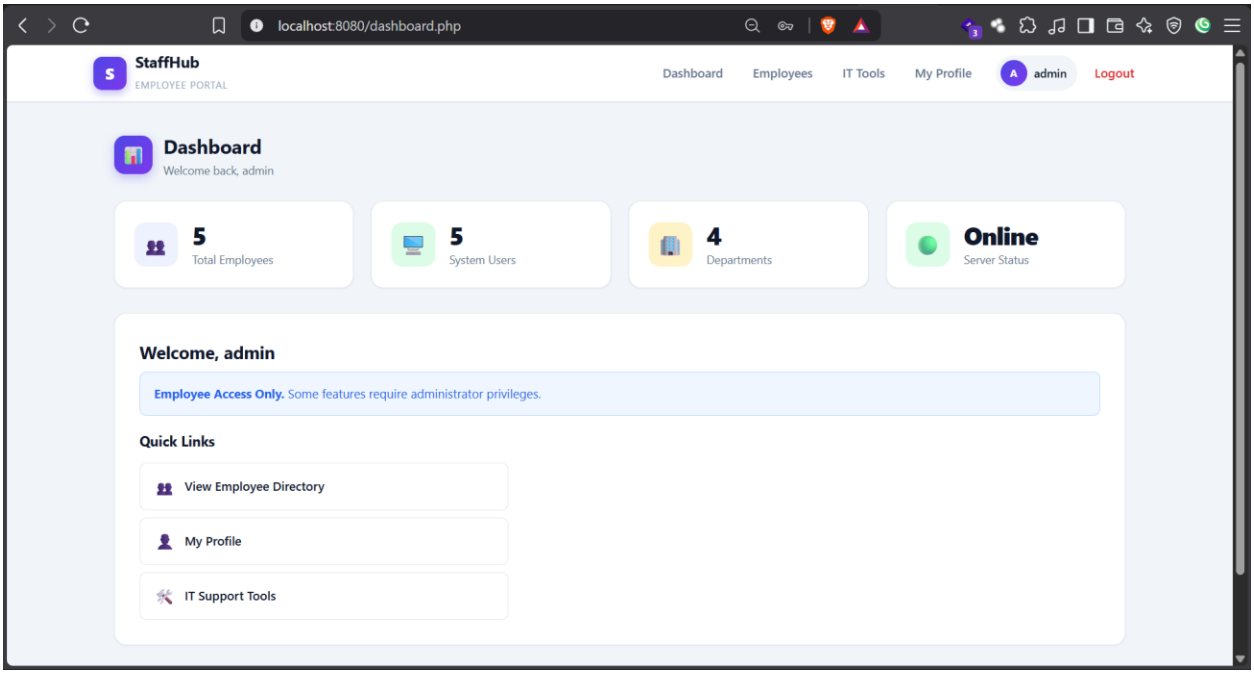

Figure 4 Successful authentication bypass using UNION SELECT payload leading to employee portal access

Step 3 - Employee Portal Reconnaissance

Action: Navigated through the authenticated employee portal and located an profile endpoint at http://localhost:8080/profile.php?id=1.

Reason: Identifying user-supplied URL parameters that dynamically fetch database records provides targets for further input validation and SQL injection testing.

Payload: Browsing to http://localhost:8080/profile.php?id=1

Explanation: The id parameter directly specifies which user profile record should be retrieved and displayed by the backend database.

Observation/Result: The page successfully rendered the profile information corresponding to user ID 1.

Screenshot:

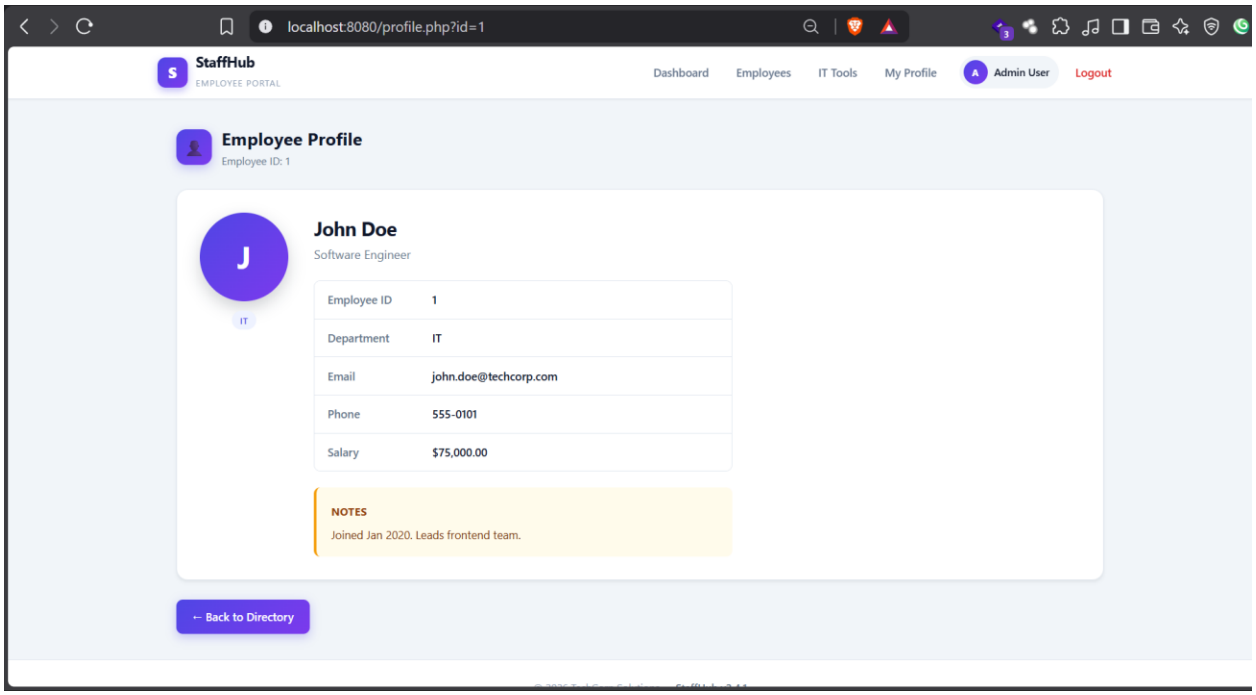

Figure 5 Employee profile page rendered via the if URL parameter

Step 4 - SQL Injection Testing on profile.php

Action: Performed incremental ORDER BY testing on the id parameter of profile.php.

Reason: To determine the column count specifically for the profile endpoint's database query, which operates independently from the login page query.

Payload/Command/Request: Appended ORDER BY positions to the id parameter (e.g., http://localhost:8080/profile.php?id=1 ORDER BY 1 through ORDER BY 10).

Explanation: Incrementing the column index identifies the structure of the SELECT statement used by profile.php.

Observation/Result: ORDER BY positions 1 through 9 processed without errors. Position 10 produced a database error, establishing that the profile.php query selects exactly nine columns (unlike the login query, which required six).

Screenshot:

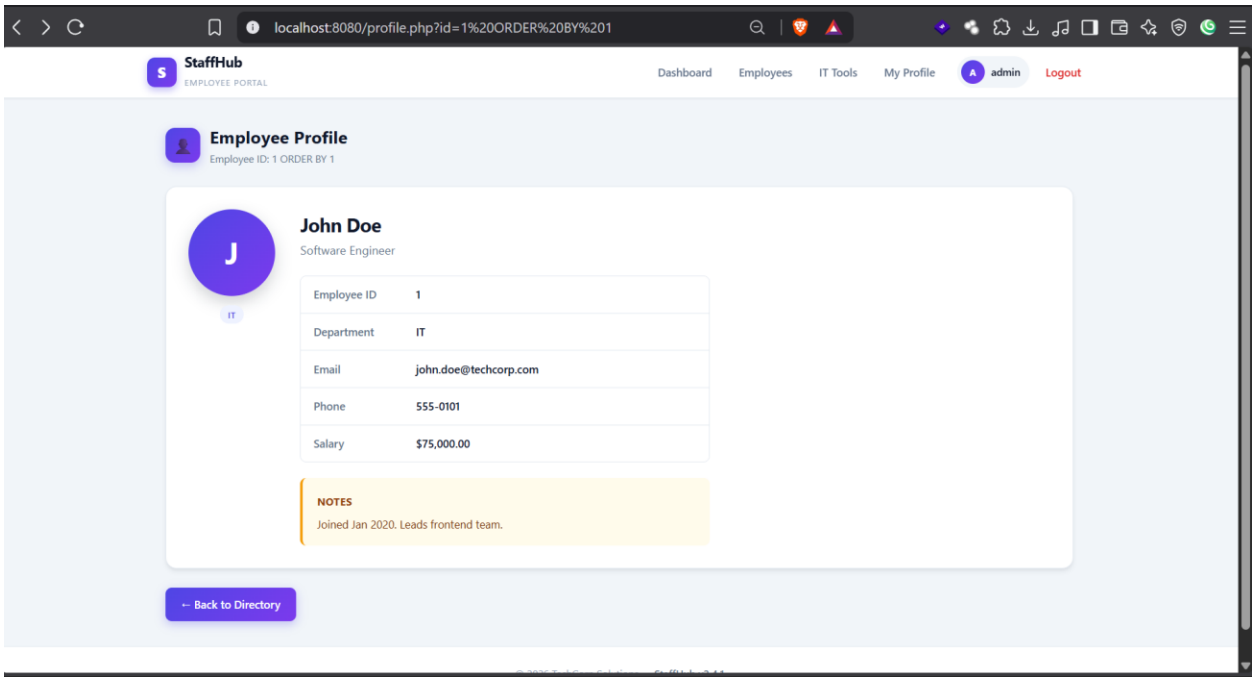

Figure 6 Successful ORDER BY Testing for Column Positions 1–9 on profile

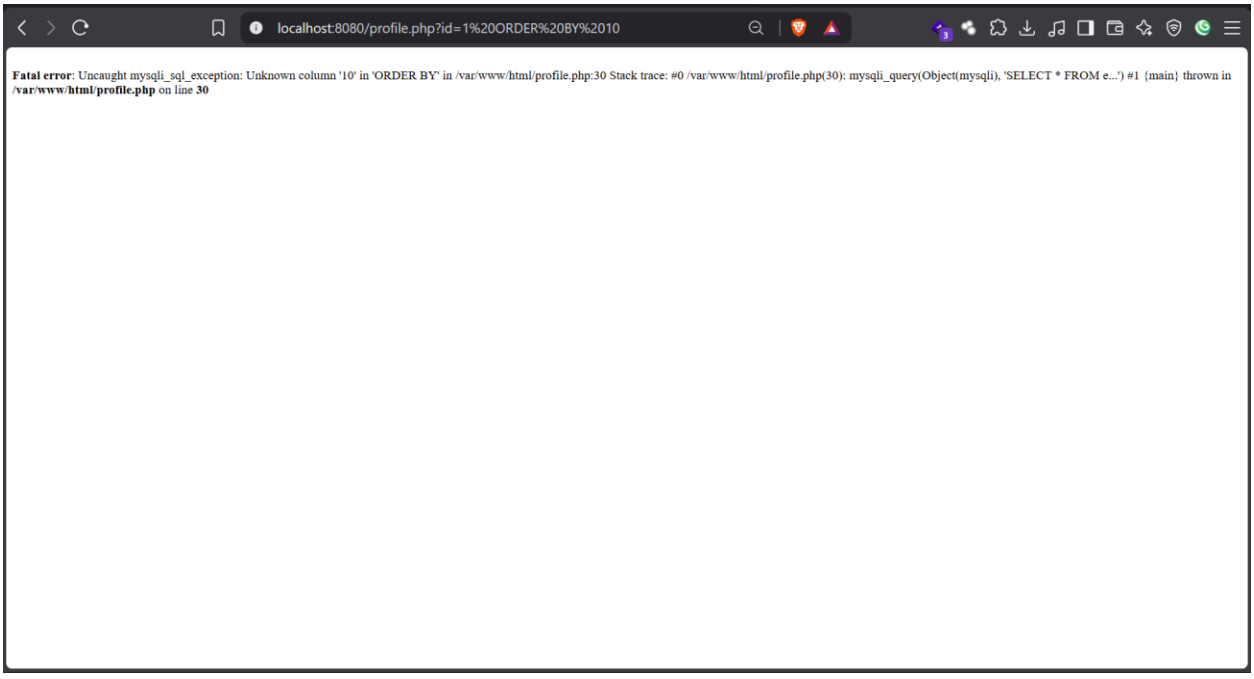

Figure 7 Database Error on profile.php Indicating an Out-of-Range Column Index at Position 10

Step 5 - UNION SELECT Proof of Concept and Data Extraction

Action: Executed a data extraction payload using a UNION SELECT statement on profile.php.

Reason: To extract user credentials directly from the users table and display them within the application interface.

Payload/Command/Request: http://localhost:8080/profile.php?id=99 UNION SELECT 1,2,3,4,5,6,7,8,(SELECT GROUP_CONCAT(CONCAT(username,0x3a,password,0x3a,role) SEPARATOR 0x0a) FROM users)#

Explanation: Breakdown of the payload components:

- **id=99:** Uses a non-existent record ID so the original query returns empty results, allowing the UNION SELECT output to render clearly.

- **UNION SELECT 1,2,3,4,5,6,7,8:** Supplies placeholder numbers for the first eight required column positions.

- **(SELECT ... FROM users):** Subquery targeting account information within the users table.

- **CONCAT(username,0x3a,password,0x3a,role):** Concatenates the username, password, and role fields into a single string separated by colons (0x3a is hexadecimal for :).

• **GROUP_CONCAT(... SEPARATOR 0x0a):** Combines multiple database rows into one text block using newlines (0x0a is hexadecimal for \n) as row separators.

• **#:** SQL comment character used to truncate the trailing portion of the server's original query. In web requests, this may require URL encoding as %23.

Observation/Result: The webpage rendered the extracted user records directly on screen, displaying usernames, passwords, and account roles.

Screenshot:

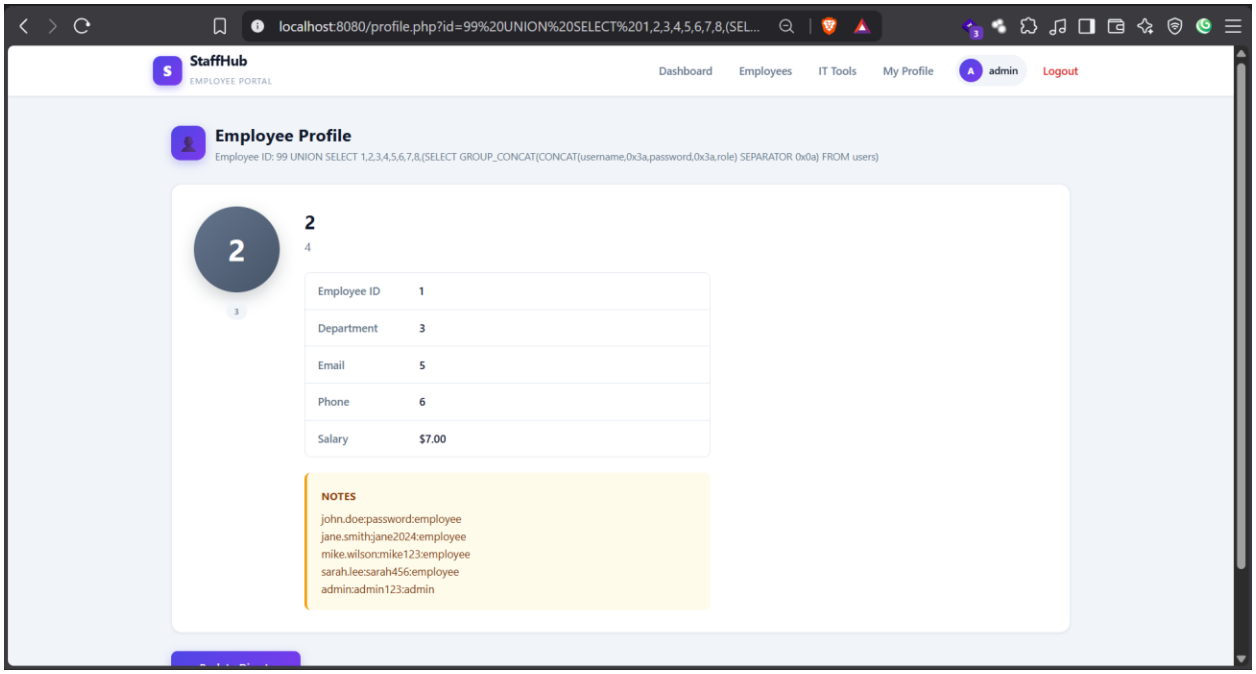

Figure 8 Extracted user accounts, passwords, and roles displayed via profile.php

Step 6 - Database Information Disclosure

Action: Analyzed the text output extracted from the users database table.

Reason: To verify the nature and sensitivity of the disclosed data.

Explanation: Evaluating the impact of data leakage resulting from improper query handling.

Observation/Result: Disclosed complete user records from the users table, including credential values, usernames, and role assignments across the application.

Screenshot:

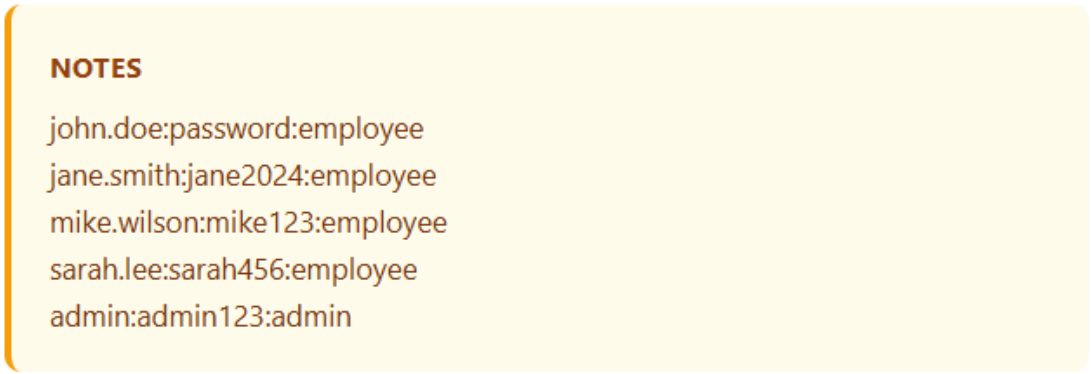

Figure 9 Extracted database contents revealing account information

Step 7 - Administrative Credential Identification

Action: Inspected the extracted records to isolate accounts possessing elevated privileges.

Reason: Identifying administrative credentials enables testing for privilege escalation paths.

Explanation: Searching the disclosed dataset for entries assigned an administrative role.

Observation/Result: Disclosed valid administrative account credentials from the extracted dataset.

Screenshot:

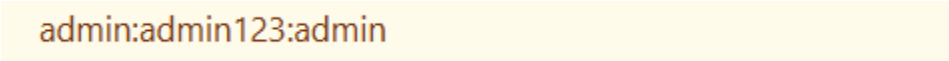

Figure 10 Administrative account credentials identified within extracted data

Step 8 - Administrative Portal Access

Action: Navigated to the administrative portal login interface and entered the extracted administrative credentials.

Reason: To verify whether the exposed credentials granted functional administrative access to the application.

Request: Submitted administrative username and password into the administrative login interface.

Result: The administrative login succeeded, providing access to the administrative control portal.

Screenshot:

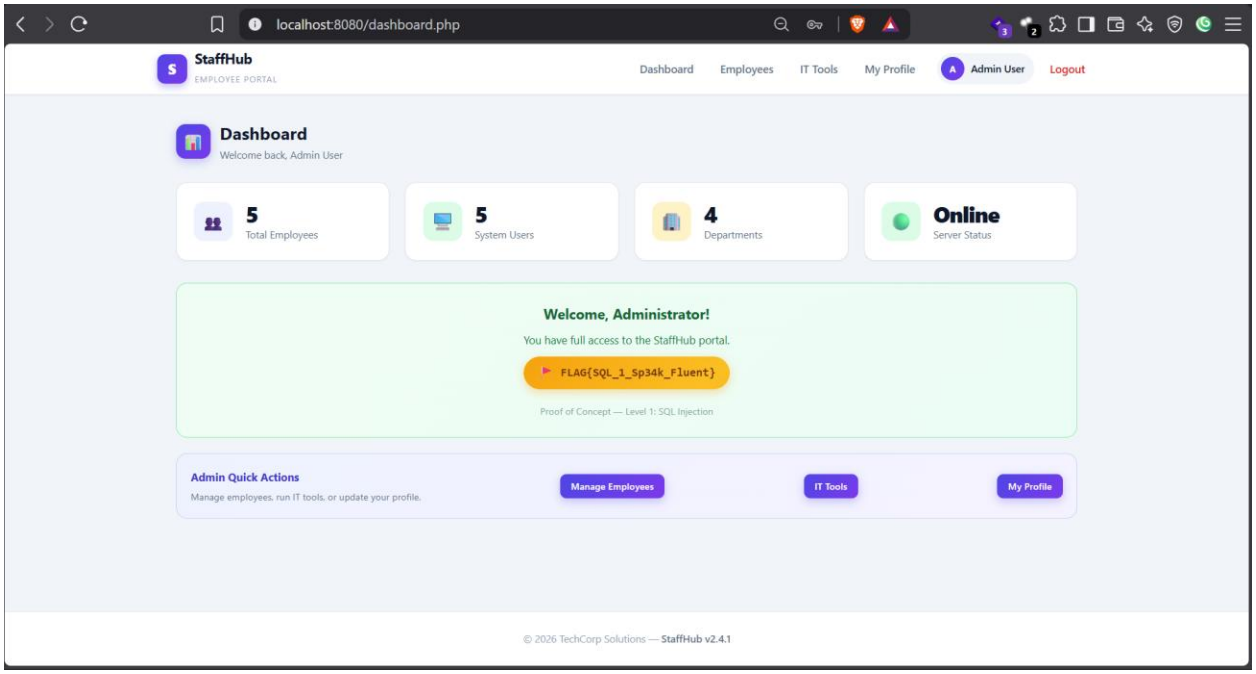

Figure 11 Successful login administrative portal using extracted credentials.

5. Proof of Exploitation

Captured Flag Code: ⚑ FLAG{SQL_1_Sp34k_Fluent}

Evidence Screenshot:

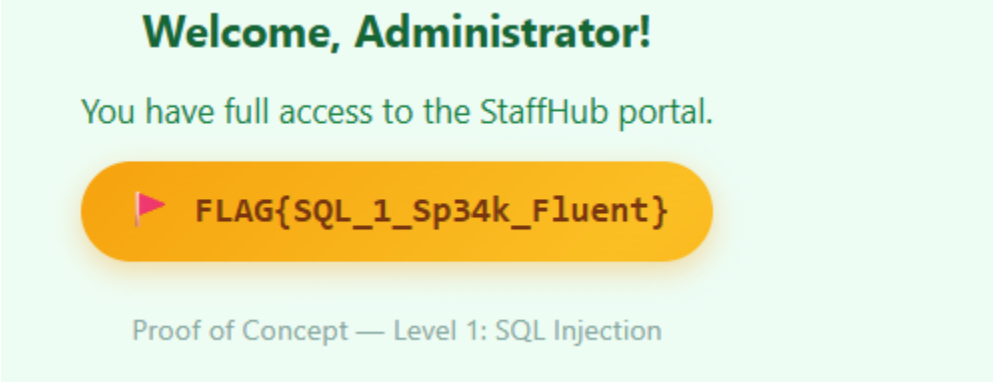

Figure 12 Captured flag

6. Business Impact

The identified SQL injection vulnerabilities present a significant security risk to the StaffHub application. During testing, an unauthenticated attacker was able to bypass authentication on index.php, extract sensitive user data via profile.php, obtain administrative credentials, and compromise administrative access.

In a operational production environment, this degree of compromise leads to severe confidentiality and integrity loss, potentially allowing unauthorized parties to access proprietary business records, alter application data, or compromise customer privacy. Such security failures can result in reputational damage, operational disruption, and non-compliance with statutory data protection regulations.

7. Remediation Recommendation

Parameterized Queries / Prepared Statements

The application should be updated to use parameterized queries (prepared statements) for all database operations. When using prepared statements, user input is bound separately from the SQL command structure, ensuring the database engine treats input strictly as data rather than executable code.

Server-Side Input Validation and Type Enforcement

Implement strict server-side validation on all incoming request parameters. For numeric parameters such as id in profile.php, enforce integer data types before passing them to internal functions .

Secure Authentication Mechanics

Authentication routines must query database records safely using prepared statements and verify credentials using secure, salted password hashing rather than dynamic string matching.

Principle of Least Privilege

Ensure the database account utilized by the web application operates with the minimum permissions necessary for routine function. The database user should not have administrative DB privileges or access to administrative schemas unless explicitly required.

Generic Error Handling

Configure the web server and database to suppress detailed error messages. Generic error screens should be displayed to end-users while detailed diagnostic logs are stored securely on the backend server for administrative review.

References

Common Weakness Enumeration , n.d. *Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection').* [Online] Available at: https://cwe.mitre.org/data/definitions/89.html [Accessed 09 08 2026].

OWASP, n.d. *A03_2021-Injection.* [Online] Available at: https://owasp.org/Top10/2021/A03_2021-Injection/ [Accessed 09 08 2026].

OWASP, n.d. *SQL Injection Bypassing WAF.* [Online] Available at: https://owasp.org/www-community/attacks/SQL_Injection_Bypassing_WAF [Accessed 10 08 2026].